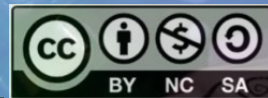


Unit Testing en Android





Testing Android Apps

1. **Unit Testing**: Validar la lògica dels mètodes del ViewModel.
2. **Instrumental UI Testing**: Testejar la interacció dels composables de la interfície d'usuari.
3. **Integration Testing**: Validar la integració de components o subsistemes.
4. **End-to-End Testing**: Testejar els fluxos complets d'interacció de l'usuari@ amb l'aplicació.
5. **UI Automator**: Proves que impliquen múltiples aplicacions o la interacció amb el sistema operatiu.
6. **Performance Testing**: Mesurar la resposta i eficiència de l'aplicació.
7. **Accessibility Testing**: Validar la disponibilitat per a usuaris amb necessitats especials.
8. **Smoke Testing**: Comprovar que les funcionalitats bàsiques funcionen.



Unit Testing (Proves unitàries)

- **Propòsit:** Provar unitats de codi aïllades, com ara mètodes o funcions dins de classes (per exemple, ViewModels o repositoris).
- **Eines comunes:** JUnit, MockK, Mockito
- **Exemples:** Comprovar la lògica del ViewModel, provar funcions de càlcul o manipulació de dades
- **Avantatges:** Són ràpides i fàcils de configurar. Es poden executar localment a la JVM, no necessiten un dispositiu o emulador Android.



Unit Testing (Proves unitàries)

- Com que haurem d'incloure moltes dependències, podem fer ús de la **BOM** de Compose per tal d'unificar les versions de les dependències carregades dins del `build.gradle.kts`.
- La BOM (Bill Of Materials) és com una "taula de versions" oficial que agrupa diverses llibreries amb la mateixa versió per tal d'evitar inconsistències.



Unit Testing (Proves unitàries)

- Fent ús de la BOM, no cal posar la versió per cada llibreria de la família (ex: Jetpack Compose).
- Evita errors de compatibilitat.
- Manté el `build.gradle.kts` més net.



Unit Testing (Proves unitàries)

- Per a usar la BOM, inclourem aquest codi a l'inici de les dependències del build.gradle.kts:

```
implementation(platform("androidx.compose:compose-bom:2024.04.01" ))  
implementation(platform(libs.androidx.compose.bom))  
androidTestImplementation(platform(libs.androidx.compose.bom))
```



Unit Testing (Proves unitàries)

- Però compte! **No totes les llibreries** són incloses dins de la taula de versionat de la BOM i, per tant, en aquestes se'ls haurà d'especificar la versió.
- Per exemple:
 - `androidx.arch.core:core-testing`
 - `androidx.lifecycle:lifecycle`
 - `androidx.test.espresso`
 - `junit:junit`



Unit Testing (Proves unitàries)

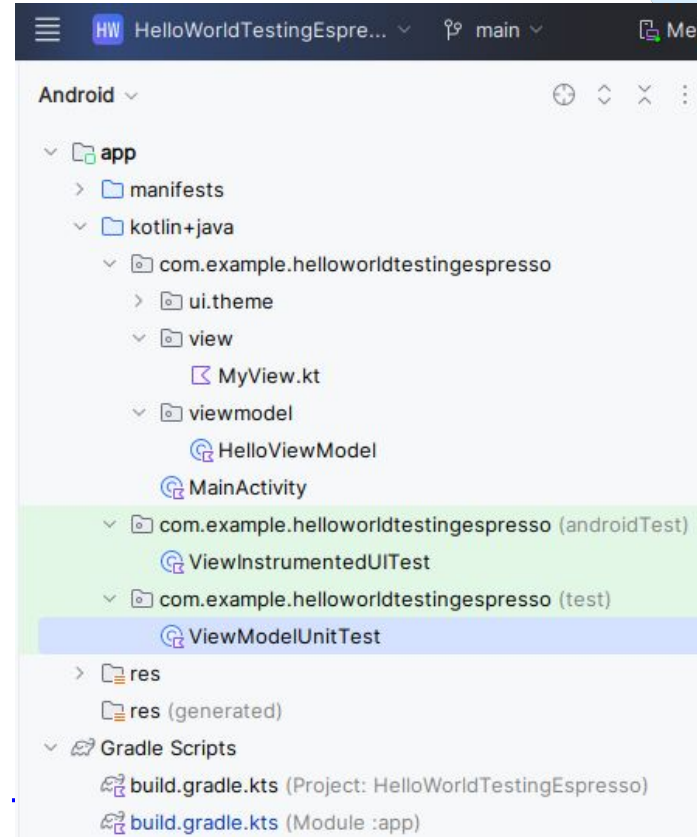
- Per poder fer unit testing, inclourem les següents dependències al **build.gradle.kts** (module:app):

```
testImplementation("androidx.arch.core:core-testing:2.2.0" )  
testImplementation(libs.junit)
```



Unit Testing (Proves unitàries)

- Estructura de carpetes del projecte incloent els arxius de test:
 - **ViewInstrumentedUITest**
 - **ViewModelUnitTest**
- Observar també el nom de la *view* i del *viewmodel*.





Unit Testing (Proves unitàries)

- L'objectiu del Unit Testing és el de **comprovar** el bon funcionament dels **mètodes del ViewModel**.
- És a dir que el valor dels **estats** de les seves **propietats** s'actualitzen correctament en funció dels mètodes que es criden.
- Com que el nostre projecte està fet seguint el patró MVVM, estem usant **LiveData** dins del ViewModel.



Unit Testing (Proves unitàries)

- Propietats/atributs d'un **ViewModel** d'exemple:

```
class HelloViewModel: ViewModel {  
    // Atributs  
    private val _textValue: MutableLiveData<String>  
    public val textValue: LiveData<String>  
  
    private val _numberOfClicks: MutableLiveData<Int>  
    public val numberOfClicks: LiveData<Int>  
  
    private val _checkBoxMulti: MutableLiveData<Boolean>  
    public val checkBoxMulti: LiveData<Boolean>  
  
    private val _counterValue: MutableLiveData<Int>  
    public val counterValue: LiveData<Int>  
  
    ...  
}
```



Unit Testing (Proves unitàries)

- Constructor:

```
constructor(): super() {  
    this._textValue = MutableLiveData<String>("")  
    this.textValue = this._textValue  
  
    this._numberOfClicks = MutableLiveData<Int>(0)  
    this.numberOfClicks = this._numberOfClicks  
  
    this._checkBoxMulti = MutableLiveData<Boolean>(false)  
    this.checkBoxMulti = this._checkBoxMulti  
  
    this._counterValue = MutableLiveData<Int>(0)  
    this.counterValue = this._counterValue  
}
```



Unit Testing (Proves unitàries)

- Mètodes:

```
fun setValue (text: String) {  
    this._textValue.value = text  
}  
  
fun clickIncrement () {  
    this._numberOfClicks.value = this._numberOfClicks.value?.plus(1)  
    updateCounterValue()  
}  
  
fun updateCounterValue () {  
    if (this._checkBoxMulti.value == false) {  
        this._counterValue.value = this._numberOfClicks.value  
    } else {  
        this._counterValue.value = this._numberOfClicks.value!! * 2  
    }  
}
```



Unit Testing (Proves unitàries)

- Mètodes:

```
fun toggleCheckBoxMulti () {  
    this._checkBoxMulti.value = !(this._checkBoxMulti.value)!!  
    updateCounterValue()  
}  
  
fun resetValues () {  
    this._textValue.value = ""  
    this._numberOfClicks.value = 0  
    this._checkBoxMulti.value = false  
    this._counterValue.value = 0  
}
```



Unit Testing (Proves unitàries)

- Quan fem Unit Testing, no ens interessa el comportament de les Views ja que estem avaluant el **comportament de les funcionalitats** definides al ViewModel. És a dir: **dels seus mètodes** que són independents de les vistes de la app.
- Farem crides als seus mètodes i aquests provocaran canvis en els valor dels estats dels seus atributs.
- Volem comprovar el bon funcionament de cada mètode de forma aïllada de la resta.



Unit Testing (Proves unitàries)

- Recordem del mòdul 0487 quan vam fer Unit Testing usant **asserts** en JUnit:

```
@Test
fun checkSumaTest () {
    assertEquals(4, 2 + 2)
}

@Test
fun checkSumaTest2 () {
    assertEquals(6, suma(3, 2))
}
```



Unit Testing (Proves unitàries)

- Dins del *framework* de testing de JUnit per a Java i Kotlin, els **asserts** més habituals són:

```
assertEquals(expected, actual)           // Comprova que dos valors siguin iguals
assertNotEquals(unexpected, actual)       // Comprova que no siguin iguals
assertTrue(condition)                     // Comprova que la condició sigui certa
assertFalse(condition)                    // Comprova que la condició sigui falsa
assertNull(value)                         // Comprova que sigui null
assertNotNull(value)                      // Comprova que no sigui null
assertSame(expected, actual)              // Comprova que siguin el mateix objecte (referència)
assertNotSame(unexpected, actual)         // Comprova que NO siguin la mateixa referència
fail("Missatge")                          // Falla el test explícitament
assertArrayEquals(unexpected, actual)     // Comprova que dos vectors siguin iguals
```



Unit Testing (Proves unitàries)

- Com que tenim **LiveData** dins del nostre VM, usarem **@get:Rule** per tal d'alterar el funcionament del *framework* JUnit i obligar-lo a actualitzar els valors del LiveData de forma síncrona:

```
class ViewModelUnitTest {  
    @get:Rule  
    val instantExecutorRule = InstantTaskExecutorRule()
```



Unit Testing (Proves unitàries)

- Usarem l'annotation `@Before` de JUnit per tal de que s'executi el codi de dins del mètode `setup()` just abans de cada unit test per tal d'inicialitzar el ViewModel:

```
private lateinit var viewModel: HelloViewModel

@Before
fun setup() {
    viewModel = HelloViewModel()
}
```



Unit Testing (Proves unitàries)

- Comprovem l'estat inicial dels valors dels atributs del ViewModel tal com els inicialitza el constructor:

```
@Test
fun initialState() {
    assertEquals("", viewModel.textValue.value)
    assertEquals(0, viewModel.numberOfClicks.value)
    assertEquals(false, viewModel.checkBoxMulti.value)
    assertEquals(0, viewModel.counterValue.value)
}
```



Unit Testing (Proves unitàries)

- Comprovem el funcionament del mètode ***setTextValue()*** del ViewModel:

```
@Test
fun checkSetTextValue () {
    viewModel.setTextValue("Maria Antonia")

    assertEquals("Maria Antonia", viewModel.textValue.value)
}
```



Unit Testing (Proves unitàries)

- Comprovem el funcionament del mètode ***clickIncrement()*** del ViewModel:

```
@Test
fun checkClickIncrement () {
    viewModel.clickIncrement()
    assertEquals(1, viewModel.numberOfClicks.value)

    viewModel.clickIncrement()
    assertEquals(2, viewModel.numberOfClicks.value)
}
```



Unit Testing (Proves unitàries)

- Comprovem el funcionament del mètode ***toggleCheckBoxMulti()*** del ViewModel:

```
@Test
fun checkToggleCheckBoxMulti () {
    viewModel.toggleCheckBoxMulti()
    assertEquals(true, viewModel.checkBoxMulti.value)

    viewModel.toggleCheckBoxMulti()
    assertEquals(false, viewModel.checkBoxMulti.value)
}
```



Unit Testing (Proves unitàries)

- Comprovem el funcionament del mètode ***resetValues()*** del ViewModel:

```
@Test
fun checkResetValues () {
    viewModel.setTextValue("Test")
    viewModel.clickIncrement()
    viewModel.clickIncrement()
    viewModel.updateCounterValue()
    viewModel.toggleCheckBoxMulti()
    viewModel.resetValues()

    assertEquals("", viewModel.textValue.value)
    assertEquals(0, viewModel.numberOfClicks.value)
    assertEquals(false, viewModel.checkBoxMulti.value)
    assertEquals(0, viewModel.counterValue.value)
}
```



Unit Testing (Proves unitàries)

ViewModelUnitTest.kt

Exemples de Unit Testing que validaran el funcionament dels mètodes del ViewModel. És independent de la UI, aquí farem tests dels mètodes i atributs del ViewModel.

See testing documentation ↗.

17

Run 'ViewModelUnitTest' Ctrl+Shift+F10

Debug 'ViewModelUnitTest'

Run 'ViewModelUnitTest' with Coverage

Modify Run Configuration...

22

@get:Rule

Run ViewModelUnitTest

Test Results 27 ms

Tests passed: 6 of 6 tests - 27 ms

```
> Task :app:javaPreCompileDebugUnitTest UP-TO-DATE
> Task :app:processDebugJavaRes UP-TO-DATE
> Task :app:compileDebugUnitTestKotlin
> Task :app:compileDebugUnitTestJavaWithJavac NO-SOURCE
> Task :app:processDebugUnitTestJavaRes UP-TO-DATE
> Task :app:testDebugUnitTest
BUILD SUCCESSFUL in 3s
22 actionable tasks: 2 executed, 20 up-to-date

Build Analyzer results available
13:57:02: Execution finished ':app:testDebugUnitTest --tests "com.example.helloworldtestingespresso'
```